Software Design Description 100

ESP32-CAM – Overview

Overview

The ESP32-CAM captures grayscale images from an OV2640 sensor and streams raw frames downstream to the ESP32-DevKitV1 over SPI DMA at 18 MHz. It operates in a wake-on-demand model: the DevKitV1 (main controller) wakes the CAM via an alarm instruction when the PIR sensor triggers, and the CAM captures continuously until told to stop.

During capture, every frame is pushed over SPI. One out of every 10 frames is buffered in PSRAM as raw data. After the DevKitV1 sends a stop command, the CAM quietly converts all buffered raw frames to JPEG with embedded timestamps, writes them to the SD card, and returns to sleep.

This design eliminates the SPI/SD pin-sharing teardown during capture entirely. SPI runs uninterrupted at full speed for the duration of the burst. SD access only happens in the post-capture idle phase.

> **Hardware:** ESP32-D0WD (ESP32-CAM board) | 8 MB PSRAM | OV2640 camera | SD card (4-bit SDMMC) | SPI master to DevKitV1

Module Hierarchy

```
main.c                      Entry point - sleep, wake handler, mode selection
|---- camera_control.c      Capture loop, PSRAM buffering, post-capture JPEG convert
|---- ov2640_config.c       Sensor configuration, proportional AEC auto-tune
|---- DMA_SPI_master.c      SPI DMA master - sends frames to DevKitV1 (MOSI)
|                            Also receives timestamp + commands from DevKitV1 (MISO)
|---- image_buffer_pool.c   PSRAM buffer pool (capture + save ring buffer)
\---- sd_storage.c          SD card mount/unmount, JPEG file writes (post-capture only)
```

Operational Flow

1. Sleep (default state)

- CAM is in deep sleep, consuming minimal power
- Woken by SPI chip select assertion from DevKitV1

2. Wake + Handshake

- DevKitV1 sends alarm instruction over SPI MISO
- Payload includes `uint32_t epoch` timestamp from RTC
- CAM stores epoch, starts `esp_timer` as local time baseline
- AEC auto-tune runs (3-5 frames, ~150ms)

3. Capture Burst

- Capture at 30 fps (QVGA 320x240 grayscale)
- **Every frame:** raw grayscale pushed to SPI DMA queue, transmitted to DevKitV1
- **Every 10th frame:** raw pixels copied to PSRAM ring buffer (no SD, no JPEG, no teardown)
- SPI runs uninterrupted at 18 MHz for entire burst
- Continues until DevKitV1 sends stop command over MISO

4. Post-Capture Save

- SPI DMA stopped
- SD card mounted (no pin conflict — SPI is idle)
- For each buffered raw frame in PSRAM:
  - JPEG encode (software, ~50-100ms per frame)
  - Filename: `img_YYYYMMDD_HHMMSS_NNN.jpg` (timestamp from epoch + elapsed)
  - Write to SD
- SD unmounted
- Return to deep sleep

Burst Capture Test Mode (`#define TEST_BURST_CAPTURE`)

- SD mounted once at boot, stays mounted
- Discards first 5 frames (AEC settle time)
- Captures 30 raw grayscale frames at max rate
- Saves to `/sdcard/img_NNNN.raw` (no JPEG, no SPI)
- Used for harvesting simulation data (fed into `sim/oct/wht_fpga.m`)

PSRAM Frame Buffering

During capture, the PSRAM serves two purposes:

Pool	Count	Size Each	Purpose
Capture framebuffers	3	76.8 KB	OV2640 DMA targets (`fb_count=3`, `GRAB_LATEST`)
Save ring buffer	~30	76.8 KB	1-in-10 frames held for post-capture JPEG conversion

Budget: 3 + 30 = 33 buffers x 76.8 KB = 2.5 MB of 8 MB PSRAM. A 10-second burst at 30fps = 300 frames, 30 saved = 2.3 MB. Plenty of headroom.

Thread-safe alloc/free via FreeRTOS mutex. Peak usage tracking for diagnostics.

OV2640 Configuration

Parameter	Value
Resolution	320x240 grayscale (QVGA)
Pixel format	GRAYSCALE
Auto exposure	Disabled after auto-tune
Auto gain	Disabled (gain=0)
Auto white balance	Disabled
Framebuffers	3 (PSRAM), `CAMERA_GRAB_LATEST`

Proportional AEC auto-tune: On wake, `ov2640_config.c` runs a closed-loop exposure tuner: 1. Set initial AEC value (default 50) 2. Capture frame, compute mean pixel brightness 3. Adjust: `aec_next = aec_current * target / mean` 4. Clamp to [1, 1200], repeat until `|mean - target| < tolerance` 5. Converges in 3-5 iterations typically

Target mean brightness: 110. Tolerance: 10.

Known limitation: In full direct sunlight, minimum AEC (aec=1) still overexposes. Deploy with shade visor or recessed housing.

SD Card

- 4-bit SDMMC mode at 40 MHz
- Mount point: `/sdcard`
- **Only accessed during post-capture phase** — never during active SPI streaming
- Timestamped filenames: `img_YYYYMMDD_HHMMSS_NNN.jpg`
- Timestamp derived from: DevKitV1 RTC epoch (received at wake) + `esp_timer` elapsed

---

Source Files

File	Purpose	Status
`src/main.c`	Entry point, sleep/wake, mode selection	Done (needs sleep/wake update)
`src/camera_control.c`	Capture loop + PSRAM buffering + post-capture JPEG	Done (needs refactor)
`src/ov2640_config.c`	Sensor config, AEC auto-tune	Done
`src/DMA_SPI_master.c`	SPI DMA master + MISO timestamp receive	Done (needs MISO handshake)
`src/image_buffer_pool.c`	PSRAM buffer pool	Done (needs save ring buffer)
`src/sd_storage.c`	SD mount/write (post-capture only)	Done (needs timestamp filenames)

Known Issues

1. **SPI/SD GPIO13 sharing** — eliminated during capture by deferring SD writes to post-capture. No longer a throughput concern.
2. **Overexposure in direct sun** — AEC bottoms out at aec=1. Mechanical shade required.
3. **Resolution mismatch** — old production mode used 640x480. Pipeline now standardized on 320x240 (QVGA) to match FPGA WHT block processing.

References

- SDD000 — Development Standards
- SDD120 — CAM to DevKit SPI Interface
- SDD200 — ESP32-DevKitV1
- ESP-IDF Camera Driver